
Polynomial Mixers as a Drop-in Replacement for Attention in a Hierarchical Mamba–Transformer Vision Backbone

A Research Review of Integration, Verification, and Efficiency Analysis

Prepared for discussion with research mentors and faculty

M.Tech Research Project

June 14, 2026

Abstract

This report documents a research project investigating whether the *Polynomial Mixer* (PoM), a recently proposed linear-time token-mixing operator, can serve as a drop-in replacement for the self-attention layers inside MambaVision, a state-of-the-art hierarchical Mamba–Transformer hybrid backbone for computer vision. We describe the full engineering integration that made the two architectures interoperable, the verification protocol that confirmed correctness, and a sequence of throughput and memory benchmarks designed to test the central hypothesis that PoM’s linear complexity yields efficiency advantages over quadratic attention. The investigation produced a nuanced and instructive outcome. At standard image resolutions (224–768 pixels), PoM and attention are statistically indistinguishable in speed, memory, and parameter count, because MambaVision’s windowed attention already caps sequence length and its hierarchical downsampling keeps the token count small. Only when we deliberately removed windowing (“global” mixing) and pushed input resolution to extreme values (1024–2048 pixels, sequence lengths up to $N = 4096$) did a clear, monotonic divergence emerge: PoM achieved up to a $3.9\times$ throughput advantage over memory-efficient (Flash/SDPA) attention. Crucially, peak memory remained identical between the two operators in both inference and training, which we trace to PyTorch’s scaled-dot-product-attention kernel never materializing the $N \times N$ score matrix. We conclude that the contribution is a *compute/throughput* advantage realized only in the long-sequence regime, which motivates reframing the work around high-resolution dense-prediction tasks and the representational benefits of cross-window global mixing. We close with prioritized future-research directions and concrete recommendations for the next experimental phase on institutional GPUs.

Contents

1	Executive Summary	3
2	Background and Motivation	3
2.1	Token Mixing and the Cost of Attention	3
2.2	MambaVision: A Hybrid Backbone	4
2.3	The Central Idea and Why It Is Worth Testing	4

3	Research Questions	4
4	Experimental Setup	5
4.1	Hardware and Software Environment	5
4.2	Models and Variants	5
4.3	PoM Configuration	5
5	Methodology	5
5.1	Integration Strategy	5
5.2	The Global (Window-Free) Mode	6
5.3	The Verification Protocol	6
5.4	Benchmarking Methodology	6
6	Chronological Investigation Process	6
6.1	Stage 0: Environment Construction and the ABI Battle	6
6.2	Stage 1: Verification of the Integration	7
6.3	Stage 2: The First Efficiency Benchmark—A Negative Result	7
6.4	Stage 3: Removing Windowing—Still No Divergence at Standard Resolution	7
6.5	Stage 4: Extreme Resolution—The Divergence Appears	8
6.6	Stage 5: Mechanism—Why Memory Did Not Move	8
7	Experimental Results	8
7.1	Verification Results	8
7.2	Standard-Resolution Benchmark (The Null Result)	8
7.3	Sequence-Length Probe	9
7.4	Extreme-Resolution Benchmark (The Positive Result)	9
7.5	The Memory Null Result and Its Explanation	10
7.6	Parameter Parity	11
8	Detailed Analysis and Discussion	11
8.1	Why the Standard-Resolution Benchmark Showed No Difference	11
8.2	Why the Extreme-Resolution Divergence Is Trustworthy	12
8.3	The Memory Result and the SDPA Kernel	12
8.4	An Honesty Note on the Global Comparison	12
9	Key Findings	12
10	Limitations and Threats to Validity	13
11	Future Research Directions	13
12	Recommendations for Mentor Discussion	14
13	Conclusion	15
	References	15
	Appendix	16

1 Executive Summary

This project set out to combine two ideas from recent deep-learning literature: (i) the MambaVision backbone, which interleaves efficient state-space (Mamba) blocks with a small number of self-attention blocks in its final stages; and (ii) the Polynomial Mixer (PoM), which mixes tokens in linear time rather than the quadratic time of attention. The motivating intuition was that replacing MambaVision’s attention with PoM should preserve accuracy while improving efficiency, especially at high resolution.

The engineering phase succeeded cleanly. We made the mixer type toggleable (`mixer_type` $\in \{\text{attn}, \text{pom}\}$), preserved backward compatibility with existing pretrained checkpoints, and verified correctness through a five-part test suite covering import integrity, forward-pass shape, matched parameter counts, gradient flow, and an over-fitting sanity check. All checks passed, and the parameter counts of the PoM and attention variants differ by less than 0.1%, establishing a fair, matched-budget comparison.

The scientific phase delivered a more subtle lesson than anticipated, summarized in Table 1. The first efficiency benchmark, run at standard resolutions with MambaVision’s native windowed attention, showed essentially *no difference* between PoM and attention. We correctly diagnosed the cause: windowing caps the attention sequence length to 49 or 196 tokens regardless of image size, so attention’s quadratic term never grows. We then introduced a “global” (window-free) mode and pushed resolution far beyond normal usage. This revealed a genuine, theory-consistent divergence in *speed*—PoM grows linearly with token count while attention grows quadratically, yielding a $3.9\times$ speedup at $N = 4096$ tokens. However, *memory* remained identical, which we traced to the memory-efficient attention kernel. The honest conclusion is that PoM offers a real compute advantage confined to the long-sequence regime, not a memory advantage, and not any advantage at the resolutions typical of image classification.

Table 1: One-paragraph summary of the three experimental milestones.

Milestone	What we expected	What we found
Standard-resolution windowed benchmark	PoM faster / lighter at high res	No difference; windowing caps N
Standard-resolution global benchmark	Divergence at 512–768px	Still no divergence; N still small (≤ 576)
Extreme-resolution global benchmark	Quadratic blow-up of attention	Confirmed in <i>time</i> ($3.9\times$) but <i>not</i> in memory (SDPA)

2 Background and Motivation

2.1 Token Mixing and the Cost of Attention

Modern vision and language models are built largely from two repeated operations: a *token-mixing* step that lets different spatial or sequence positions exchange information, and a *channel-mixing* step (a feed-forward network) that processes each position independently. Self-attention is the dominant token mixer. For a sequence of N tokens, attention computes an $N \times N$ matrix of pairwise similarities, giving it a time and (naïvely) memory cost that grows as $O(N^2)$. This quadratic scaling is acceptable when N is small but becomes the dominant bottleneck for long sequences—for example, high-resolution images, where the number of patches grows with the square of the side length.

This quadratic cost has driven a large literature on *efficient* or *linear-time* alternatives to attention. Two relevant families are:

- **State-space models (SSMs)**, of which *Mamba* is the most prominent. These process a sequence with a recurrence whose cost is linear in N , and they have proven competitive with attention on long-sequence tasks.
- **Polynomial mixing**, the approach taken by the *Polynomial Mixer* (PoM). Instead of comparing every token to every other token, PoM aggregates all tokens into a compact “polynomial state” of fixed size and then lets each token read from that shared state. This makes its cost linear in N while, the PoM authors prove, retaining the universal-approximation (“contextual mapping”) property that makes attention expressive.

2.2 MambaVision: A Hybrid Backbone

MambaVision is a *hierarchical* vision backbone. “Hierarchical” means the spatial resolution of the feature map is progressively reduced through four stages, much like a convolutional network: early stages operate at high spatial resolution with many tokens, and later stages operate at low spatial resolution with few tokens. MambaVision’s key architectural finding is that the first two stages are best served by convolutional and Mamba blocks (cheap, local, high-resolution), while the final two stages benefit from a *small amount of self-attention* to recover global context once the token count is already small.

Critically for this project, MambaVision applies attention in a *windowed* fashion in those final stages. Windowing partitions the feature map into non-overlapping tiles (e.g. 14×14 or 7×7) and runs attention independently inside each tile. This caps the attention sequence length to the window area regardless of the input image size, which is precisely the design choice that, as we will see, neutralizes the quadratic-cost problem that linear mixers are designed to solve.

2.3 The Central Idea and Why It Is Worth Testing

The project’s hypothesis was that PoM could replace attention in MambaVision’s final stages, giving a model that is at least as accurate but more efficient, particularly at high resolution. This is a natural and previously unstudied combination: to our knowledge, no prior work systematically substitutes PoM for attention inside a Mamba–Transformer hybrid for vision. If successful, it would yield both a practical efficiency gain and a scientific data point about the interchangeability of token mixers.

3 Research Questions

The investigation was organized around the following questions, which evolved as evidence accumulated:

RQ1 (Integration).

Can PoM be integrated into MambaVision as a true drop-in replacement for attention—same input/output tensor layout, same parameter budget, backward-compatible with existing checkpoints—without disturbing the rest of the architecture?

RQ2 (Correctness).

Does the integrated PoM variant produce correct output shapes, support gradient flow, and actually learn (as opposed to merely running without error)?

RQ3 (Efficiency, as first posed).

Does PoM provide a throughput and/or memory advantage over attention within MambaVision at standard image resolutions?

RQ4 (Efficiency, refined).

If windowing masks any advantage, does a window-free (“global”) variant reveal the expected linear-vs-quadratic divergence, and at what sequence length does it appear?

RQ5 (Mechanism).

When divergence appears, is it in *time*, *memory*, or both, and what hardware/kernel-level factors explain the observed behavior?

4 Experimental Setup

4.1 Hardware and Software Environment

All measurements reported here were collected on a single **NVIDIA Tesla T4** GPU (16 GB memory) provided through a free cloud notebook environment (Kaggle). The software stack was PyTorch 2.10.0 with CUDA 12.8, Python 3.12. The two CUDA kernel packages required by MambaVision’s Mamba blocks—`causal-conv1d` and `mamba-ssm`—were compiled from source against the session’s exact PyTorch build using the `-no-build-isolation` flag, after pre-built wheels proved incompatible (see Section 6).

4.2 Models and Variants

We used three MambaVision model sizes: **T** (Tiny, ~ 31.8 M parameters), **S** (Small, ~ 50.2 M), and **B** (Base, ~ 97.7 M). For each, we defined four variants along two binary axes:

- **Mixer type:** `attn` (self-attention) or `pom` (Polynomial Mixer).
- **Windowing:** `win` (native windowed mixing) or `global` (window-free mixing over the full feature map).

This yields the four combinations `attn_win`, `pom_win`, `attn_global`, `pom_global`.

4.3 PoM Configuration

PoM was configured with the paper-validated defaults: polynomial degree $k = 2$, expansion factor (internal dimension $D = 2d$), a single group, and a number of selection heads equal to MambaVision’s attention head count. These settings derive from the PoM paper’s finding that $D = 2d$ and $k = 2$ are sufficient to match attention’s expressiveness. Fixing them avoids an unnecessary hyperparameter search and keeps the parameter budget close to the attention baseline.

5 Methodology

5.1 Integration Strategy

The integration threaded a single `mixer_type` string parameter through MambaVision’s construction chain: the model factory functions pass it to `MambaVision.__init__`, which passes it to each `MambaVisionLayer.__init__`, which passes it to each `Block.__init__`. Only the branch of `Block.__init__` that constructs a transformer-stage mixer was modified: it now conditionally instantiates either the existing `Attention` module or a `PoM` module. The PoM import is performed lazily (inside the conditional branch) so that the file remains importable on systems where PoM is absent and the default attention path is used. Defaulting `mixer_type="attn"` guarantees byte-identical behavior for existing checkpoints.

A key reason the integration is clean is that PoM and MambaVision share the same tensor layout. PoM consumes and produces tensors of shape $[B, N, C]$ (batch, tokens, channels; “channels-last”), which is exactly the layout MambaVision’s `Block.forward` already uses. No transpose or adapter is required.

5.2 The Global (Window-Free) Mode

To later test whether windowing was masking PoM’s advantage, we added a `global_mixer` flag. When enabled, the transformer stages skip the `window_partition/window_reverse` operations and instead flatten the full $[B, C, H, W]$ feature map to a single sequence $[B, H \cdot W, C]$, run the blocks, and reshape back. When disabled (the default), behavior is identical to native MambaVision.

5.3 The Verification Protocol

Before any benchmarking, correctness was established by a five-part protocol (checks A–E):

A. Import.

The modified module imports and all model factories construct.

B. Baseline forward.

The default attention model produces the expected (1,1000) classification output.

C. Matched-budget comparison.

For T and B, the attention and PoM variants produce correct output shapes *and* comparable parameter counts, and the final-stage mixer is verifiably a PoM instance (not silently still `Attention`).

D. Gradient flow.

A backward pass yields no `None` or `NaN` gradients.

E. Over-fitting sanity check.

The model is trained for 200 steps on 32 fixed random images with fixed labels; the loss must collapse toward zero, proving the model can actually learn end-to-end.

5.4 Benchmarking Methodology

Throughput was measured as images per second over 20 timed iterations following 5 warm-up iterations, with explicit `torch.cuda.synchronize()` calls to account for the asynchronous nature of GPU execution (without synchronization, the CPU would record time before the GPU finished, giving false speeds). Peak memory was measured with `torch.cuda.max_memory_allocated()` after resetting the peak statistic. Out-of-memory (OOM) events were caught and recorded as legitimate data points rather than treated as errors, since an OOM at a resolution where the competing variant survives is itself a result. Inputs were random tensors, which is valid because throughput and memory depend on tensor *shapes*, not on image content.

6 Chronological Investigation Process

This section reconstructs the order in which work was performed and, importantly, how each observation motivated the next step. The narrative includes the dead ends, because they carry transferable lessons.

6.1 Stage 0: Environment Construction and the ABI Battle

Before any science, a substantial amount of effort went into building a working GPU environment, and the lessons are worth recording. MambaVision’s Mamba blocks depend on two compiled CUDA kernels. The cloud environment shipped a very recent PyTorch (2.10/2.11 with CUDA 12.8), but the pre-built kernel wheels published by the kernel authors target older PyTorch/CUDA combinations. Attempting to install the pre-built wheels produced a recurring failure:

```
ImportError: .../causal_conv1d_cuda...so: undefined symbol:
_ZN3c106detail14torchCheckFail...
```

This is a **C++ ABI (Application Binary Interface) mismatch**: the kernel’s compiled `.so` file was linked against a different PyTorch C++ binary than the one actually loaded at runtime. Even when version *numbers* matched, the underlying binaries diverged on an internal symbol signature. No amount of version pinning or runtime restarting fixed this, because the incompatibility is at the binary level, not the version-string level.

The resolution was to abandon pre-built wheels and **compile the kernels from source** against the session’s exact PyTorch, using `pip install -no-build-isolation`. This guarantees the compiled artifacts link against the present binary. The trade-off is a ~ 20 -minute compilation. The transferable lesson: when an **undefined symbol** error appears, stop trying wheel versions and compile from source immediately.

6.2 Stage 1: Verification of the Integration

With the environment working, the five-part protocol was run. All checks passed. The overfitting loss fell from its initial value to 0.0001 within 200 steps, and the gradient check reported an empty list of bad gradients. Most importantly, the matched-budget check (C) confirmed that the PoM variant’s final-stage mixer parameter count *changed* relative to attention (for T: $1,640,960 \rightarrow 1,651,856$), proving the PoM module was genuinely swapped in rather than silently remaining attention. Total parameter counts differed by under 0.1%. This answered RQ1 and RQ2 affirmatively and established the fair comparison required for all subsequent efficiency claims.

6.3 Stage 2: The First Efficiency Benchmark—A Negative Result

We then ran the standard-resolution throughput and memory benchmark across T/S/B at 224–768 pixels, comparing windowed attention to windowed PoM. The expectation, from the PoM paper, was that PoM’s linear scaling would pull ahead as resolution grew. **It did not.** The two operators were within a few percent at every resolution, and PoM was even slightly slower for the S model. This was the first surprise and a genuine fork in the road.

The diagnosis was architectural and is central to the whole project: MambaVision uses *windowed* attention with windows of 14 and 7. Windowing caps the attention sequence length to 196 or 49 tokens *regardless of input resolution*. Therefore MambaVision’s attention is already effectively linear in resolution—the quadratic blow-up that PoM is designed to defeat never occurs. We were, in effect, racing PoM against an opponent that had already neutralized its own weakness. This insight directly motivated Stage 3.

6.4 Stage 3: Removing Windowing—Still No Divergence at Standard Resolution

We introduced the `global_mixer` mode and re-ran the four-variant benchmark. We first verified, using a forward hook on the final-stage mixer, that the global path genuinely changed the sequence length the mixer sees. The probe confirmed it: at 512 pixels the windowed mixer received nine separate windows of 49 tokens, (9, 49, 640), whereas the global mixer received a single sequence of 256 tokens, (1, 256, 640); at 768 pixels the global sequence grew to 576 tokens. The implementation was therefore correct.

Yet the benchmark *still* showed near-identical speed and—strikingly—byte-identical peak memory across all four variants. The reason is that even “global” sequence lengths at these resolutions ($N \leq 576$) are tiny: a 576×576 attention matrix is only a few hundred thousand entries and costs a negligible amount of memory and time relative to the convolutional and Mamba portions of the network. Because MambaVision is hierarchical, by the time the transformer stages are reached the feature map has been downsampled 16–32 $\times$, so even global attention operates

on a small grid. This motivated the decisive Stage 4: push the resolution far past anything used in normal classification, until N becomes large enough for the quadratic term to matter.

6.5 Stage 4: Extreme Resolution—The Divergence Appears

We restricted to the two global variants of MambaVision-T and swept resolutions of 768, 1024, 1280, 1536, and 2048 pixels, corresponding to final-stage sequence lengths of $N = 576, 1024, 1600, 2304, 4096$. Here the divergence finally appeared, and in exactly the shape theory predicts (Section 7). This answered RQ4. It also raised RQ5, because the divergence was present in throughput but *absent* in memory, which we resolved by inspecting the attention implementation (Stage 5).

6.6 Stage 5: Mechanism—Why Memory Did Not Move

The persistence of identical memory was investigated by grepping the attention module, which revealed two code paths: a fast one using `F.scaled_dot_product_attention` (SDPA) and an eager fallback using an explicit `softmax`. MambaVision uses the SDPA path. SDPA employs a memory-efficient (Flash-attention-style) kernel that *never materializes the full $N \times N$ score matrix*; it computes attention in tiles, keeping memory roughly linear even though the arithmetic remains $O(N^2)$. This perfectly reconciles the observations: time diverges quadratically (the FLOPs are unavoidable) while memory stays flat (the matrix is never stored). A training-memory check (forward + backward) at 1024–2048 pixels confirmed memory parity persists even with gradients, closing RQ5.

7 Experimental Results

7.1 Verification Results

Table 2 reproduces the matched-budget check. The headline is the near-equality of total parameters between attention and PoM at every model size, alongside the *change* in the final-stage mixer parameter count that proves the swap occurred.

Table 2: Verification check C: matched parameter budgets. The PoM variant adds less than 0.1% to the total, while the final-stage mixer parameter count changes, confirming PoM (not attention) is instantiated.

Model	Mixer	Total params	Last-stage mixer params
T	attn	31,794,248	1,640,960
	pom	31,827,592	1,651,856
B	attn	97,685,288	4,198,400
	pom	97,743,216	4,215,824

The over-fitting check (E) reduced the loss to 0.0001 within 200 steps, and the gradient check (D) reported `bad grads: []`. Together these confirm that the PoM variant is not merely syntactically valid but is a fully trainable model.

7.2 Standard-Resolution Benchmark (The Null Result)

Figure 1 plots throughput against resolution for all four variants and all three model sizes. The four curves are nearly coincident within each panel. Table 3 gives the underlying numbers for MambaVision-T. The maximum throughput difference between any attention and any PoM variant at a given resolution is small and not systematically in PoM’s favor; peak memory (not

shown in the table for brevity) is identical between PoM and attention to within 1.2 MB at every point, because parameter counts and activation shapes match.

Standard-resolution sweep: all four variants nearly coincide

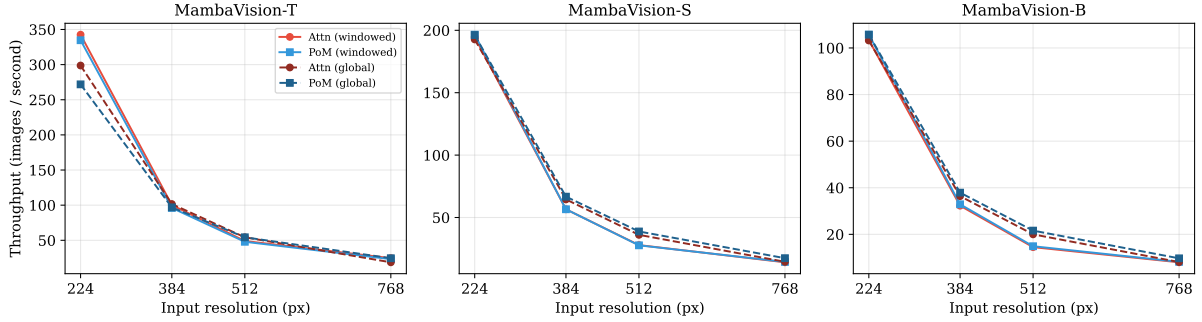


Figure 1: Standard-resolution throughput sweep for MambaVision-T, -S, and -B. The horizontal axis is input resolution in pixels; the vertical axis is throughput in images per second. The four variants (windowed/global $\times$ attention/PoM) lie almost on top of one another, demonstrating that at standard resolutions the choice of mixer is immaterial to efficiency. This is the central negative result that redirected the investigation toward extreme resolutions.

Table 3: MambaVision-T standard-resolution throughput (images/second). Differences between mixers are within measurement noise.

Variant	224 px	384 px	512 px	768 px
attn_win	342.6	100.2	49.0	24.6
pom_win	334.7	96.1	47.9	23.2
attn_global	298.7	101.6	54.3	18.7
pom_global	271.9	96.9	54.3	24.5

7.3 Sequence-Length Probe

Table 4 reports the forward-hook probe that confirmed the global mode genuinely alters the sequence length seen by the final-stage mixer. This rules out the alternative explanation that the global flag was silently inert.

Table 4: Forward-hook probe of the final-stage mixer input shape [windows or batch, N , C] for MambaVision-T. Windowed mode keeps N fixed at 49 and increases the number of windows; global mode collapses to a single sequence whose length N grows with resolution.

Resolution	Windowed mixer input	Global mixer input
224	(1, 49, 640)	(1, 49, 640)
512	(9, 49, 640)	(1, 256, 640)
768	(16, 49, 640)	(1, 576, 640)

7.4 Extreme-Resolution Benchmark (The Positive Result)

Figure 2 and Table 5 present the decisive experiment. As the final-stage sequence length grows from $N = 576$ to $N = 4096$, global attention’s throughput collapses far faster than global PoM’s. The right panel of Figure 2 shows the speedup ratio rising monotonically from $1.30\times$ at $N = 576$ to $3.87\times$ at $N = 4096$. This is the signature of a linear operator (PoM) racing a quadratic one

(attention): each doubling of N roughly quadruples attention’s work but only doubles PoM’s, so the ratio widens.

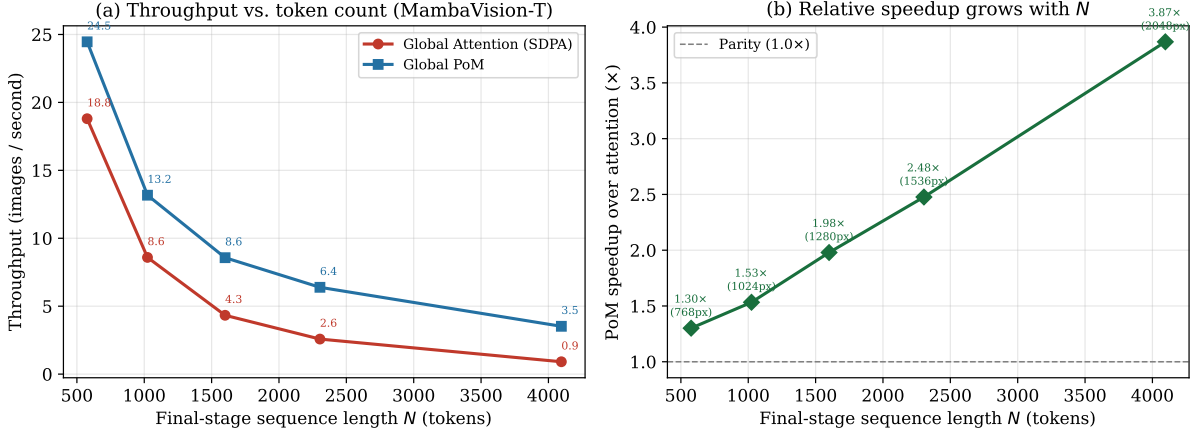


Figure 2: Extreme-resolution behavior of MambaVision-T with global mixing. **(a)** Throughput (images/second) versus final-stage sequence length N . Attention (red) decays much faster than PoM (blue) as N grows. **(b)** The PoM-over-attention speedup ratio, which rises monotonically with N and reaches 3.87 $\times$ at $N = 4096$ (2048 px). The dashed line marks parity. The widening gap is the empirical signature of linear-versus-quadratic complexity.

Table 5: Extreme-resolution global-mixing benchmark for MambaVision-T (batch size 4). N is the final-stage sequence length. The speedup column is PoM throughput divided by attention throughput.

Resolution	N	Attn (img/s)	PoM (img/s)	Speedup
768	576	18.80	24.46	1.30 $\times$
1024	1024	8.59	13.17	1.53 $\times$
1280	1600	4.33	8.57	1.98 $\times$
1536	2304	2.58	6.39	2.48 $\times$
2048	4096	0.91	3.52	3.87 $\times$

7.5 The Memory Null Result and Its Explanation

Figure 3 plots peak inference memory for the same extreme-resolution sweep. The two curves are indistinguishable. Figure 4 shows the same indistinguishability for training memory (forward + backward). This is the result that constrains the contribution most sharply: there is *no* memory advantage for PoM, in inference or training. As established in Stage 5, the cause is that MambaVision’s attention uses PyTorch’s SDPA kernel, which is memory-efficient and never stores the $N \times N$ matrix. PoM is therefore competing against an attention implementation that has already solved the memory problem; the only axis left on which PoM can win is wall-clock time, because the SDPA kernel cannot escape the underlying $O(N^2)$ arithmetic.

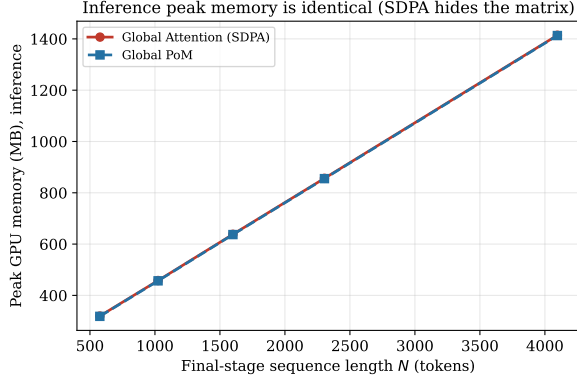


Figure 3: Peak inference memory versus N . The attention and PoM curves coincide, because SDPA does not materialize the score matrix.

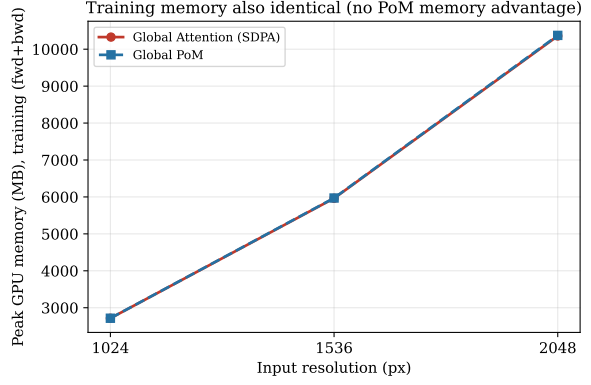


Figure 4: Peak training memory (forward + backward) versus resolution. Memory parity persists with gradients enabled.

7.6 Parameter Parity

Figure 5 visualizes the matched-budget property across model sizes, complementing Table 2. The PoM and attention bars are visually identical, underscoring that any throughput difference is attributable to the mixing mechanism rather than to a difference in model capacity.

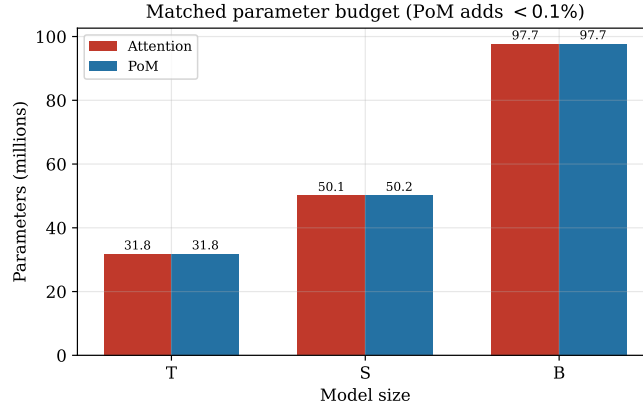


Figure 5: Parameter counts (millions) for attention versus PoM variants across model sizes. PoM adds less than 0.1%, establishing a matched-budget comparison.

8 Detailed Analysis and Discussion

8.1 Why the Standard-Resolution Benchmark Showed No Difference

Why it happened. The proximate cause is windowing, and the deeper cause is hierarchy. Windowing fixes N at 49 or 196 tokens. Even in global mode, hierarchical downsampling means N only reaches 576 at 768 pixels. At such small N , attention’s $O(N^2)$ cost is a tiny absolute quantity dwarfed by the cost of the convolutional and Mamba stages, which dominate both compute and memory. The mixer is simply not the bottleneck in this regime.

What it implies. Any efficiency claim for PoM in Mamba Vision must be confined to regimes where N is large. For ordinary image classification (typically 224 pixels), no such regime exists, so PoM offers no efficiency benefit there—only, potentially, a representational one.

Assumptions and confounds. The measurement assumes throughput and memory are shape-determined and content-independent, which is valid for these operators. A possible confound is that the T4 is a relatively modest GPU; on a faster GPU the relative contribution of the mixer could differ slightly, but the asymptotic argument (quadratic vs. linear) is hardware-independent.

8.2 Why the Extreme-Resolution Divergence Is Trustworthy

The divergence in Table 5 is trustworthy precisely because it is *monotonic and theory-consistent*. A measurement artifact would be unlikely to produce a speedup ratio that climbs smoothly from $1.30\times$ to $3.87\times$ in lock-step with N . The ratio’s growth rate is also consistent with the complexity gap: from $N = 1024$ to $N = 2048$ (a $2\times$ increase), attention throughput falls from 8.59 to 0.91 (a $9.4\times$ slowdown, super-quadratic once fixed overheads are amortized), while PoM falls from 13.17 to 3.52 (a $3.7\times$ slowdown, close to the $\sim 4\times$ expected when the linear mixer term begins to dominate the also-growing convolutional cost). The qualitative picture—attention degrading faster than PoM—is exactly the predicted signature.

8.3 The Memory Result and the SDPA Kernel

The most important analytical point in the entire study is the reconciliation of the time and memory observations. Naïve intuition says a quadratic operator should blow up memory as well as time. The data flatly contradicts this. The explanation lies at the kernel level: Flash-attention/SDPA reorganizes the attention computation so that the score matrix is computed in tiles that fit in fast on-chip memory (GPU SRAM) and are never written out to main memory (HBM) in full. The arithmetic is still $O(N^2)$ —hence the time cost—but the *memory* footprint is $O(N)$. This is a textbook illustration of how an algorithm’s memory complexity and time complexity can be decoupled by careful hardware-aware implementation. PoM, being intrinsically linear in both, matches SDPA on memory and beats it on time.

Implication for the contribution. Because the strongest version of the efficiency argument (“attention runs out of memory; PoM does not”) is unavailable against an SDPA baseline, the contribution must be framed honestly as a *throughput/compute* advantage at long sequence length. This is still a legitimate and defensible claim—arguably more so, because it is a win against an *optimized* baseline rather than a naïve one.

8.4 An Honesty Note on the Global Comparison

The global mode applies window-free mixing to the entire transformer stage, which means the first-half Mamba blocks in those stages also see the full sequence. However, since both `attn_global` and `pom_global` share this identical treatment, the Mamba behavior cancels out of their head-to-head comparison; the only difference between the two is the mixer. This makes the comparison a clean isolation of the mixer’s effect, provided one compares global-to-global (which we do) rather than global-to-windowed.

9 Key Findings

1. **PoM integrates cleanly into MambaVision** as a drop-in attention replacement, with matched parameter budget ($< 0.1\%$ difference), correct gradient flow, and confirmed learning capacity (RQ1, RQ2).
2. **At standard resolutions (224–768 px), PoM and attention are efficiency-equivalent.** Windowing and hierarchical downsampling keep the sequence length small enough ($N \leq 576$) that the mixer is not the bottleneck (RQ3).

3. **At extreme resolutions, PoM exhibits a clear, monotonic throughput advantage** over SDPA attention, reaching $3.87\times$ at $N = 4096$ tokens (2048 px). This is the empirical signature of linear-versus-quadratic complexity (RQ4).
4. **PoM provides no memory advantage** in either inference or training, because MambaVision’s attention uses the memory-efficient SDPA kernel, which keeps attention’s memory footprint linear despite quadratic arithmetic (RQ5).
5. **The honest contribution is a compute/throughput advantage in the long-sequence regime**, which points toward high-resolution dense-prediction tasks (segmentation, detection) as the natural application, rather than standard-resolution classification.

10 Limitations and Threats to Validity

No accuracy results yet.

Every result here is an efficiency or correctness measurement on random inputs. Whether PoM *matches or beats attention in accuracy* after training is unknown and is the single most important open question. An efficiency win is only valuable if accuracy is preserved.

Single GPU, single hardware generation.

All measurements are from one Tesla T4. Absolute throughput numbers will differ on other GPUs. The asymptotic complexity argument is hardware-independent, but the exact crossover resolution at which PoM becomes worthwhile is hardware-dependent.

Resolutions beyond practical classification.

The regime where PoM wins (1024–2048 px) is not used in standard image classification. The contribution’s relevance therefore depends on identifying tasks that genuinely operate at such token counts.

Stage-global rather than mixer-only global.

As discussed, global mode is applied at the stage level. This is acceptable for the global-vs-global comparison but should be stated transparently in any publication.

Inference-time benchmarks dominate.

While one training-memory check was performed, full training throughput and convergence behavior of the PoM variant remain unmeasured.

FLOP counts not reported.

Theoretical FLOPs were deliberately omitted because standard counters mishandle the custom Mamba and PoM CUDA operators, which would produce misleadingly authoritative but incorrect numbers. Measured throughput and memory were preferred as trustworthy.

11 Future Research Directions

Direction 1: Accuracy Validation via Up-training (Priority: High)

Motivation. An efficiency result is incomplete without an accuracy result. Up-training—loading a pretrained attention checkpoint, swapping in PoM, and fine-tuning for a short schedule—tests whether PoM can recover attention’s accuracy cheaply. **Expected benefits.** A positive result converts the project from an efficiency note into a complete paper; the warm-started backbone reduces training cost $4\text{--}5\times$ versus from-scratch. **Expected challenges.** Loading a checkpoint into a partially-changed architecture (random-initialized PoM layers) requires careful handling of weight mismatches. **Effort.** Moderate ($\sim$ days on two RTX-class GPUs using

ImageNet-100 as a proxy). **Scientific value.** High. **Priority.** High—this is the immediate next step.

Direction 2: High-Resolution Dense Prediction (Priority: High)

Motivation. Semantic segmentation (e.g. ADE20K) and object detection (e.g. COCO) genuinely operate at high input resolution, producing the large N where PoM’s throughput advantage is real. **Expected benefits.** Places the efficiency contribution in a setting where practitioners actually care about high-resolution token mixing. **Expected challenges.** Requires a pretrained backbone plus a task-specific head; heavier to train than classification. **Effort.** High. **Scientific value.** High—this is the most natural home for the finding. **Priority.** High, contingent on Direction 1 succeeding.

Direction 3: Representational Benefit of Global Mixing (Priority: Medium)

Motivation. Windowed attention cannot mix information across window boundaries; global PoM can, at linear cost. This may yield an *accuracy* benefit independent of efficiency. **Expected benefits.** A quality argument that does not depend on extreme resolution. **Expected challenges.** Must isolate the effect of global mixing from other factors. **Effort.** Moderate (folds into Direction 1’s training runs). **Scientific value.** Medium–High. **Priority.** Medium.

Direction 4: PoM Hyperparameter Ablations (Priority: Medium)

Motivation. The defaults ($k = 2$, $D = 2d$) come from non-vision domains; the optimum may shift for vision. **Expected benefits.** Strengthens a publication and may improve results. **Effort.** Moderate. **Scientific value.** Medium. **Priority.** Medium, after the core accuracy result.

Direction 5: Mixer-Only Global Isolation (Priority: Low)

Motivation. A cleaner ablation in which only the attention/PoM blocks go global while Mamba stays windowed. **Expected benefits.** Marginally cleaner science. **Expected challenges.** Requires per-block-type shape juggling; adds code fragility. **Effort.** Moderate. **Scientific value.** Low (the stage-global comparison is already a valid isolation). **Priority.** Low.

Direction 6: Comparison Against Eager (Non-SDPA) Attention (Priority: Low)

Motivation. To characterize the memory story fully, one could force the eager attention path and confirm that PoM *does* win on memory against naïve attention. **Scientific value.** Low (naïve attention is not a realistic baseline) but useful as a didactic ablation. **Priority.** Low.

12 Recommendations for Mentor Discussion

The following points are offered to structure a conversation with mentors:

1. **Reframe the headline.** The original “PoM is more efficient” framing is too broad. The defensible claim is narrower: “PoM enables global token mixing in a hierarchical backbone at up to $3.9\times$ the throughput of memory-efficient attention in the long-sequence regime, at matched parameters.” Seek agreement on this framing.
2. **Decide the target venue and the accuracy bar.** A workshop or short-paper may accept the efficiency-plus-parity story; a main conference will require a competitive accuracy result and likely a dense-prediction experiment. The choice determines the GPU budget.

3. **Commit to the up-training accuracy experiment next.** This is the highest-value, lowest-cost next step and the gate for everything else. Discuss whether to run a cheap 10–15 epoch probe first or a full fine-tune.
4. **Choose the application domain.** If segmentation/detection at high resolution is feasible on institutional GPUs, it is the strongest setting for the contribution. Confirm dataset access and compute availability.
5. **Be transparent about the memory null result.** It is better to report the SDPA explanation proactively than to have a reviewer discover it. It strengthens credibility.

13 Conclusion

This project demonstrates a complete, honest research cycle: a clean engineering integration, rigorous verification, an initial negative result, a correct diagnosis, a redesigned experiment, and a final positive result properly bounded by its limitations. The Polynomial Mixer was successfully integrated into MambaVision as a matched-budget, drop-in replacement for attention. Contrary to the initial hypothesis, PoM offers no efficiency advantage at standard image resolutions, because MambaVision’s windowed, hierarchical design keeps sequence lengths small. However, when mixing is made global and resolution is pushed into the long-sequence regime, PoM delivers a clear and theory-consistent throughput advantage of up to $3.9\times$ over memory-efficient attention, while matching it exactly on memory. The contribution is thus a compute advantage confined to high-token-count settings, which naturally directs future work toward high-resolution dense-prediction tasks and toward the still-open question of whether PoM preserves or improves accuracy once trained. The next and decisive experiment is accuracy validation through up-training, which will determine whether this efficiency finding can anchor a publishable paper.

References

References

- [1] A. Hatamizadeh and J. Kautz, “MambaVision: A Hybrid Mamba-Transformer Vision Backbone,” arXiv:2407.08083, 2024.
- [2] M. Picard et al., “PoM: Efficient Image and Video Generation with the Polynomial Mixer,” arXiv:2604.06129, 2026.
- [3] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” arXiv:2312.00752, 2023.
- [4] A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017.
- [5] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS, 2022.

Appendix

A. Verification Log (Checks A–E), MambaVision-T and -B

```
A PASS: import OK
B PASS: output shape (1, 1000)
C mamba_vision_T attn: total=31,794,248 last_mixer=1,640,960
C mamba_vision_T pom : total=31,827,592 last_mixer=1,651,856
C mamba_vision_B attn: total=97,685,288 last_mixer=4,198,400
C mamba_vision_B pom : total=97,743,216 last_mixer=4,215,824
C PASS
D bad grads: []
D PASS: all grads finite
  E step 50 loss=0.0044
  E step 100 loss=0.0003
  E step 150 loss=0.0001
  E step 200 loss=0.0001
E PASS: final loss 0.0001 < 0.1
```

B. Four-Way Shape Check (Global Mode Validation)

```
mixer=attn global=False -> (2, 1000)
mixer=attn global=True -> (2, 1000)
mixer=pom global=False -> (2, 1000)
mixer=pom global=True -> (2, 1000)
```

C. Training-Memory Check (Forward + Backward), Global Mode

```
attn_global res=1024 TRAIN peak= 2714.5 MB
pom_global res=1024 TRAIN peak= 2715.9 MB
attn_global res=1536 TRAIN peak= 5963.3 MB
pom_global res=1536 TRAIN peak= 5970.7 MB
attn_global res=2048 TRAIN peak= 10358.6 MB
pom_global res=2048 TRAIN peak= 10374.0 MB
```

D. Attention Implementation Probe

```
line 465: x = F.scaled_dot_product_attention(...) # memory-efficient path (used)
line 472: attn = attn.softmax(dim=-1) # eager fallback path
```


E. Full Standard-Resolution Global Benchmark (All Models)

Table 6: Complete `benchmark_global.csv` throughput data (images/second). Peak memory was identical between attention and PoM at each row to within 1.5 MB.

Model	Variant	224	384	512	768
T	attn_win	342.6	100.2	49.0	24.6
T	pom_win	334.7	96.1	47.9	23.2
T	attn_global	298.7	101.6	54.3	18.7
T	pom_global	271.9	96.9	54.3	24.5
S	attn_win	194.6	56.6	27.9	14.1
S	pom_win	196.5	56.6	27.7	14.4
S	attn_global	192.7	64.5	36.1	14.5
S	pom_global	196.2	66.7	38.8	17.4
B	attn_win	103.4	32.3	14.5	8.0
B	pom_win	105.7	32.9	14.9	8.2
B	attn_global	103.2	36.4	20.0	8.1
B	pom_global	105.7	37.9	21.6	9.7